

Best Practice for eForms

www.therefore.net
© 2024 Therefore Corporation

All rights reserved. No parts of this work may be reproduced in any form or by any means - graphic, electronic, or mechanical, including photocopying, recording, taping, or information storage and retrieval systems - without the written permission of the publisher.

Windows® is a registered trademark of Microsoft Corporation in the United States and/or other countries.

Any other 3rd Party products that are referred to in this document may be either trademarks and/or registered trademarks of the respective owners. The publisher and the author make no claim to these trademarks.

While care has been taken in the preparation of this document, the publisher and the author assume no responsibility for errors or omissions, or for damages resulting from the use of information contained in this document.

VERSION: 2024 - 01

Table of Contents

1. Introduction	5
2. eForms Advanced Features	6
2.1 Advanced Logic	6
2.1.1 Limit Select Box Component to Only One Check	6
2.1.2 Set Value	9
2.1.3 Logic with Hidden Component	12
2.1.4 Button to Trigger Logic	15
2.2 Calendar	19
2.2.1 Date is Week Date	19
2.2.2 Restrict Date	20
2.3 CSS	21
2.3.1 Field CSS	21
2.3.2 Full CSS	22
2.3.3 Custom CSS Class	22
2.3.4 Panel CSS	24
2.3.5 Advanced CSS	25
2.3.6 Scripting for Dynamic Color Changing	29
2.3.7 Color-Changing Background Using CSS	30
2.4 Grid	30
2.4.1 Layout	30
2.4.2 Sum & Calculations	32
2.4.3 Save Collapse	32
2.4.4 Hide Column in Grid Render for End User	33
2.5 Image	36
2.5.1 Change HTML According to Trigger	36
2.5.2 Base 64	37
2.6 Page Break	38
2.6.1 In a PDF	38
2.6.2 In HTML	41
2.7 PDF	42
2.7.1 PDF customization	42
2.8 Pre-populate field	43
2.8.1 Basic	43

2.8.2	Advanced	43
2.8.3	Error Handling	44
2.8.4	Escaping Special Characters	45
2.9	Web Call	45
2.9.1	Call Webservice from eForm	45
2.9.2	Call the Therefore™ WebAPI	46
2.9.3	Get Data Back (Response) from REST Call	48
2.9.4	Select Component from Web Service	49
2.10	Other	50
2.10.1	Dependent Lookup	50
2.10.2	Do Not Save PDF	52
2.10.3	PDF Page Width	53
2.10.4	Using script to Hide/Disable	54
2.10.5	Hide field on Multiple Conditions	55
2.10.6	Hide in PDF	56
2.10.7	Wizard eForm: Carry Data Between Pages	56

1. Introduction

The purpose of this document is to outline and explain advanced functionality of Therefore™ eForms.

2. eForms Advanced Features

2.1 Advanced Logic

2.1.1 Limit Select Box Component to Only One Check

The **Select Boxes Component** can be found under **Basic Components**. Navigate to the **Validation** tab.

Here, can use the built-in feature to define the number of boxes that can be selected by entering a **Minimum checked number** and/or a **Maximum checked number** of boxes that need to be checked before the form can be submitted.

Select Boxes Component

[Display](#)[Data](#)[Validation](#)[API](#)[Conditional](#)[Logic](#)[Layout](#)

☐ Required ?

Validate On ?

Change x ▼

Custom Error Message ?

Custom Error Message

Minimum checked number ?

Maximum checked number ?

Minimum checked error message ?

Maximum checked error message ?

+ Custom Validation

+ JSONLogic Validation

To change the selected box automatically and unselect the previous one, use **Logic**.

7

Select Boxes Component

[Display](#)
[Data](#)
[Validation](#)
[API](#)
[Conditional](#)
[Logic](#)
[Layout](#)

Advanced Logic

1 Advanced Logic Configured

Logic Name *

Check

Trigger

Container

Type

JavaScript

Text Area

1 debugger;

2 result = data.selectBoxes2.test3;

'row', 'data', and 'component' variables are available. Return 'result'.

Preview

Select Boxes

☐ Test
 ☐ Test1
 ☐ Test2
 ☐ Test3

Save

Cancel

Remove

Actions

1 actions

Actions

Action Name *

Show

Type

Value

Value (Javascript)

```
1 value = 'test3';
```

'row', 'data', 'component', and 'result' variables are available. Return the value.

Save Action Cancel

+ Add Action

Save Logic Cancel

+ Add Logic

2.1.2 Set Value

In this scenario, the user wants to set a hidden number component to 0 or 1, depending on whether a checkbox is checked or not. The change should only happen if the previous value is changed. Since the previous value is not typically available for this logic, you can save it to a hidden component called "hiddenSavedValue", which can be added to the form. When the value in "hiddenSavedValue" differs from the current value, this indicates that it was changed. This triggers the script to update the dependent number value.

A **Number Component** can be added under **Basic Components**.

Number Component

Display
Data
Validation
API
Conditional
Logic
Layout

Label
Number

☐ Hide Label

Label Position
Top

Placeholder
Placeholder

Description
Description for this field.

Tooltip
To add a tooltip to this field, enter text here.

Error Label
Error Label

Custom CSS Class
Custom CSS Class

Tab Index
Tab Index

☒ Clear Value When Hidden
☐ Hidden
☐ Hide Input
☐ Disabled
☐ Initial Focus

Preview
Number

Save
Cancel
Remove

Navigate to the **Logic** tab and add the script.

Number Component

Display Data Validation API Conditional **Logic** Layout

Advanced Logic

1 Advanced Logic Configured

Logic Name *

Trigger Rule

Trigger

Container

Type

Javascript

Text Area

```
1 if (data.checkbox2 !== undefined && data.checkbox2 !== data.hiddenSave
2 {
3   result = true;
4   data.hiddenSavedValue = data.checkbox2;
5 }
6 else
7 {
8   result = false;
9 }
```

Preview

Number

Save Cancel Remove

Next, set a value to a field triggered by a checkbox.

Actions

1 actions

Actions

Action Name *

Set The Value

Type

Value

Value (Javascript)

```

1  if (data.checkbox2 === true)
2    value = 1;
3  else
4    value = 0;

```

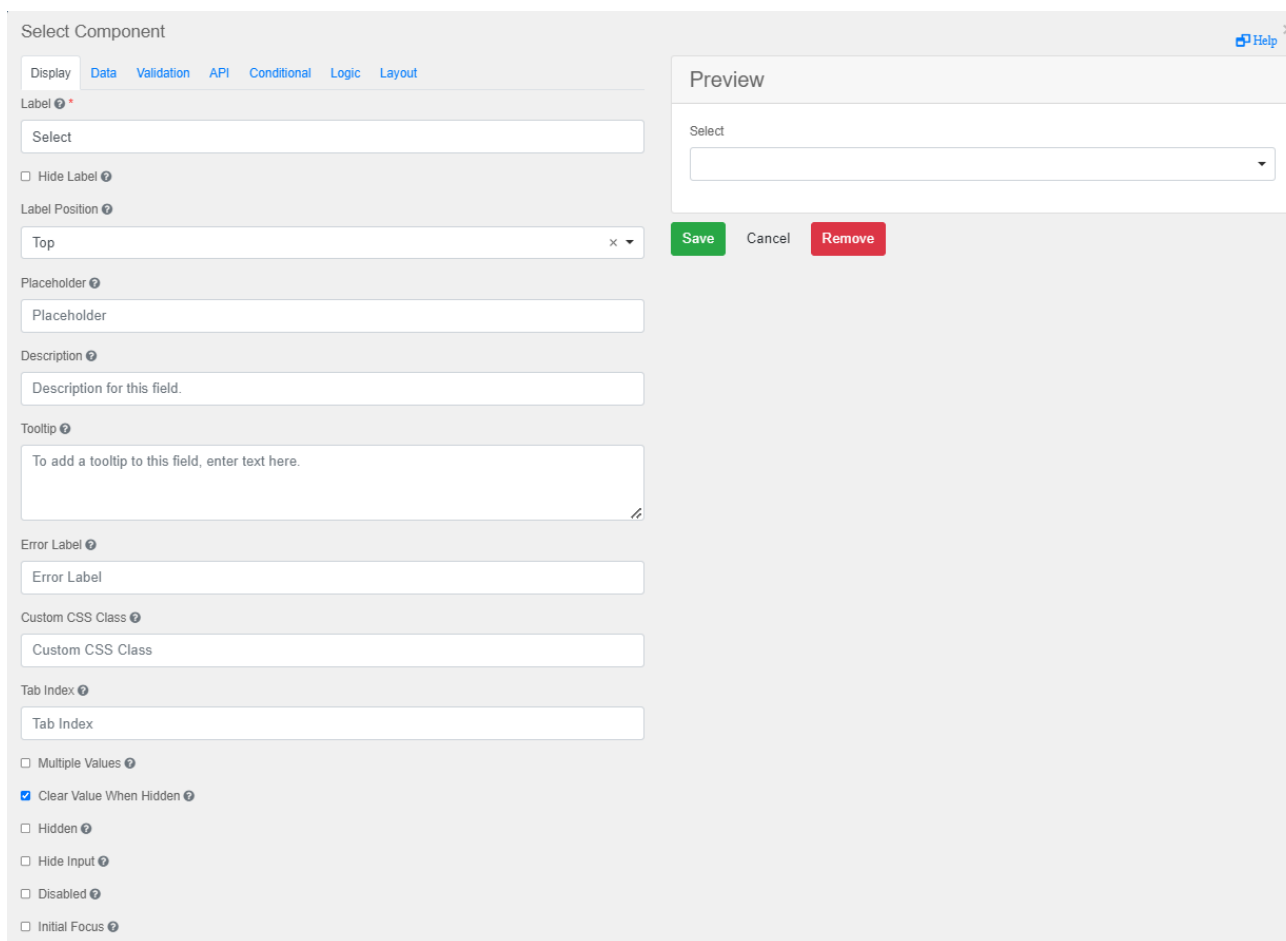
'row', 'data', 'component', and 'result' variables are available. Return the value.

Save Action Cancel

2.1.3 Logic with Hidden Component

This logic will reset the value of a **Select Component** based on a trigger (checkbox).

Add a Select component. It can be found under **Basic Components**.



Select Component

Display Data Validation API Conditional Logic Layout

Label *

Select

☐ Hide Label

Label Position

Top x ▾

Placeholder

Placeholder

Description

Description for this field.

Tooltip

To add a tooltip to this field, enter text here.

Error Label

Error Label

Custom CSS Class

Custom CSS Class

Tab Index

Tab Index

☐ Multiple Values

☒ Clear Value When Hidden

☐ Hidden

☐ Hide Input

☐ Disabled

☐ Initial Focus

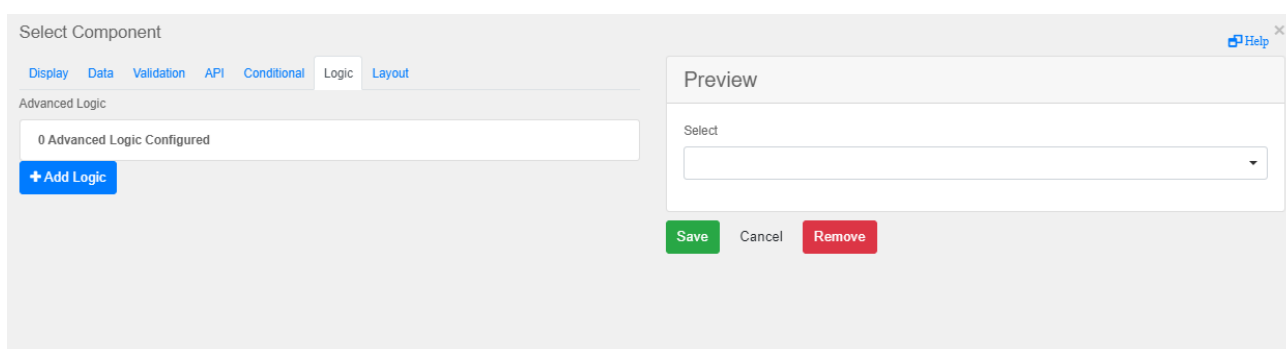
Preview

Select

Select

Save Cancel Remove

Navigate to the **Logic** tab.



Select Component

Display Data Validation API Conditional Logic Layout

Advanced Logic

0 Advanced Logic Configured

+ Add Logic

Preview

Select

Select

Save Cancel Remove

In this scenario, the customer wants to clear a Select Component field in case a different dependent field is cleared again and had a value previously. This follows the same concept as the previous section

([Set Value](#)): Keep the old value in a hidden component, and if this changes, update the dependent components.

Select Component

[Display](#)
[Data](#)
[Validation](#)
[API](#)
[Conditional](#)
[Logic](#)
[Layout](#)

Advanced Logic

1 Advanced Logic Configured

Logic Name *

Trigger

Trigger

Container

Type

Javascript x ▾

Text Area

```

1  if (data.selectA !== undefined && data.selectA != data.hidden)
2  {
3      result = true;
4  }
5  else
6  {
7      result = false;
8  }
9  data.hidden=data.selectA;

```

Preview

Select B

▾

Save

Cancel

Remove

'row', 'data', and 'component' variables are available. Return 'result'.

Actions

1 actions

Actions

Action Name *

ClearField

Type

Value x ▼

Value (Javascript)

1 value = '';

'row', 'data', 'component', and 'result' variables are available. Return the value.

Save Action Cancel

2.1.4 Button to Trigger Logic

This is the recommended method to trigger a logic from a button.

Note: There is no method to force the UI to see that a button has been activated/clicked. This means that an automated action cannot be triggered from the button reaction.

Navigate to the Display tab of the **Button Component** and select **Event** from the dropdown menu under **Action**. Under **Button Event**, enter **SetText**.

Button Component

Display

Data

Validation

API

Conditional

Logic

Layout

Label ? *

Submit

☐ Hide Label ?

Label Position ?

Top

×

▼

Placeholder ?

Placeholder

Action ?

Event

×

▼

☐ Show Validations ?

Button Event ?

SetText

Next, navigate to the **Logic tab** and define the **Action** associated with the SetText button event. To do so, select **Event** from the dropdown menu under **Type** and enter **SetText** as the **Event Name**. To proceed to the next step, click **Add Action**.

Button Component

Display Data Validation API Conditional **Logic** Layout

Advanced Logic

0 Advanced Logic Configured

Logic Name *

Trigger

Trigger

Container

Type

Event

Event Name

SetText

The event that will trigger this logic. You can trigger events externally or via a button.

Actions

0 actions

+ Add Action

Save Logic Cancel

+ Add Logic

Enter an **Action Name** for your Action and select **Value** from the dropdown menu under **Type**. Define the value using a script. Click **Save Action** to conclude.

Actions

0 actions

Actions

Action Name *

SetValue

Type

Value

x ▼

Value (Javascript)

```
1 value = "Test";
2
```

row", "data", "component", and "result" variables are available. Return the value.

Save Action

Cancel

2.2 Calendar

2.2.1 Date is Week Date

To check if a selected day is a weekday or to limit the choice of date based on weekday the script below can be used.

Custom Error Message ?

Please do not select a weekend day!

Custom Validation

The following variables are available in all scripts.

input	The value that was input into this component
form	The complete form JSON object
submission	The complete submission object.
data	The complete submission data object.
row	Contextual "row" data, used within DataGrid, EditGrid, and Container components
component	The current component JSON
instance	The current component instance.
value	The current value of the component.
moment	The moment.js library for date manipulation.
_	An instance of Lodash .
utils	An instance of the FormioUtils object.
util	An alias for "utils".

```

1 var isWeekday = ((input.getDay() !== 0) && (input.getDay() !== 6));
2 valid = isWeekday ? true : false;

```

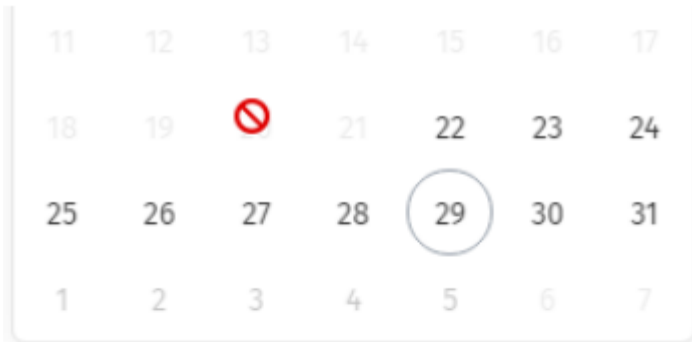
```

var isWeekday = ((input.getDay() !== 0) && (input.getDay() !== 6));
valid = isWeekday ? true : false;

```

2.2.2 Restrict Date

To restrict calendar availability in the **Date/Time Component** the following of code can be utilized.



Date / Time Component

Display
Date
Time
Data
Valid

☒ Enable Date Input ?

Minimum Date ?

moment().subtract(7, 'days')

Maximum Date ?

moment().add(7, 'days');

2.3 CSS

2.3.1 Field CSS

Some eForm components support CSS directly in the **Layout** tab.

Text Field Component

Display Data Validation API Conditional Logic **Layout**

HTML Attributes ⓘ

Attribute Name	Attribute Value
style	background-color: #BED500

+ Add Attribute

Preview

Text Field

Save Cancel Remove

Below is a non-exhaustive list of fields that support (or don't support) CSS:

CSS Supported	CSS NOT Supported
Text field	Date
Table Lookup	Text Area
Email	Currency
Number	
Password	
Phone Number	
Address field	

Below is a non-exhaustive list of CSS supported in the Layout tab:

```
width: 100px;
height: 20px;
position: relative;
background-color: #BED500;
border-style: solid;
border-width: 10px;
border-color: #595959;
color: white;
padding: 2px 2px;
text-decoration: line-through;
vertical-align: center;
display: block;
font-size: 16px;
margin: 4px 4px;
cursor: pointer;
border-radius: 24px;
```

background-size: 60%;
background-repeat: no-repeat;
background-position: 50% 10%;

Text Field Component

Display Data Validation API Conditional Logic Layout

HTML Attributes ?

Attribute Name	Attribute Value
style	width: 100px; height: 20px; position: relative; background-c

+ Add Attribute

Preview

Text Field

Save Cancel Remove

2.3.2 Full CSS

To get full CSS inside any eForm, you need to use an **HTML Element component** with a style bracket under **Content**. The HTML Element component can be found under **Advanced**.

Example:

Content ?

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <style>
5 .disclaimer{
6   color:red; text-align:center;
7 }
8 </style>
9 </head>
10 </html>

```

2.3.3 Custom CSS Class

To get the same CSS on multiple components, use the custom CSS feature.

Example:

Content ?

```
1 <!DOCTYPE html>|
2 <html>
3 <head>
4 <style>
5 .sample-class
6 {
7   font-family: "Comic Sans MS", cursive, sans-serif;
8   font-size: 25px;
9   letter-spacing: 2px;
10  word-spacing: 2px;
11  color: #000000;
12  font-weight: normal;
13  text-decoration: none;
14  font-style: normal;
```

☐ Refresh On Change ?

```
<!DOCTYPE html>
<html>
<head>
<style>
.sample-class
{
font-family: "Comic Sans MS", cursive, sans-serif;
font-size: 25px;
letter-spacing: 2px;
word-spacing: 2px;
color: #000000;
font-weight: normal;
text-decoration: none;
font-style: normal;
font-variant: normal;
text-transform: none;
}
</style>
</head>
</html>
```

To use your custom CSS class, enter the name of the class in the **Custom CSS class field** of a component.

Tooltip ?

To add a tooltip to this field, enter text here.

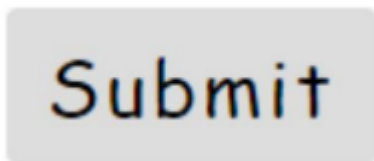
Error Label ?

Error Label

Custom CSS Class ?

sample-class

This sample changes the button to look as pictured below:



2.3.4 Panel CSS

Some CSS will affect existing components, not exclusively HTML components.

```
.panel{
  box-shadow:5px 5px 0px 0px #a6cff5
}
.panel-heading{
  background-color:#f0f0f0;
  color:#595959;
}
.panel-title{
  font-size: 170%;
}
```

This code will create (see screenshot below):

- A shadow around 2 sides of all “panels” in your eForm (5px)
- Title section with grey background
- Panel title with larger font (170%)

Therefore
o n l i n e

Personnal Information

First Name

Email Address

ex: myname@example.com

Last Name

Phone Number

ex: (555) 555-5555

Work in the office

How many days you request to work from the office

Department

2.3.5 Advanced CSS

An eForm that needs advanced visual design or advanced, dynamic reactions can be updated using a full CSS setup.

HTML Element Component

Display

Data

Validation

API

Conditional

Logic

Layout

Label

HTML

☐ Hide Label

Label Position

Top

x

HTML Tag

p

CSS Class

CSS Class

Attributes

Attribute	Value	

+ Add Another

Preview

Upload section

Save

Cancel

Remove

An HTML component can support full CSS:

Content ?

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <style>
5 .panel{
6   box-shadow : 5px 5px 1px 1px #BED500;
7 }
8 .square-button {
9   width: 100%;
10  height: auto;
11  position:relative;
12  background-color: #BED500;
13  border-style: solid;
14  border-width: 5px;
```

☐ Refresh On Change ?

Placeholder ?

Placeholder

Example:

This setup will create

- 2 types of titles
- Shadow over the panel
- 1 type of button
- 1 mouse-over effect on button

```
<!DOCTYPE html>
<html>
<head>
<style>
.panel{
  box-shadow : 5px 5px 1px 1px #BED500;
}
.square-button {
  width: 100%;
  height: auto;
  position: relative;
  background-color: #BED500;
  border-style: solid;
  border-width: 5px;
  border-color: #595959;
  color: white;
  padding: 2px 2px;
  text-decoration: none;
  vertical-align: center;
  display: block;
  font-size: 16px;
  margin: 4px 4px;
  cursor: pointer;
  border-radius: 24px;
  background-size: 60%;
  background-repeat: no-repeat;
  background-position: 50% 10%;
}
.button2-secondary: hover
{
  background-color: #595959;
  border-color: #595959;
}
.button-text {
  position: relative;
  bottom: 1%;
  width: 100%;
}

.title1 {text-align: center;
border-style: solid;
border-color: #595959;
border-radius: 24px;
background-color: #595959;
color: #BED500;
font-family: "Tahoma", serif;
border-width: 5px;
padding: 30px;
}
.title2 {text-align: center;
border-style: solid;
border-color: #595959;
border-radius: 12px;
background-color: ;
color: #BED500;
font-family: "Tahoma", serif;
border-width: 5px;
padding: 10px;
}

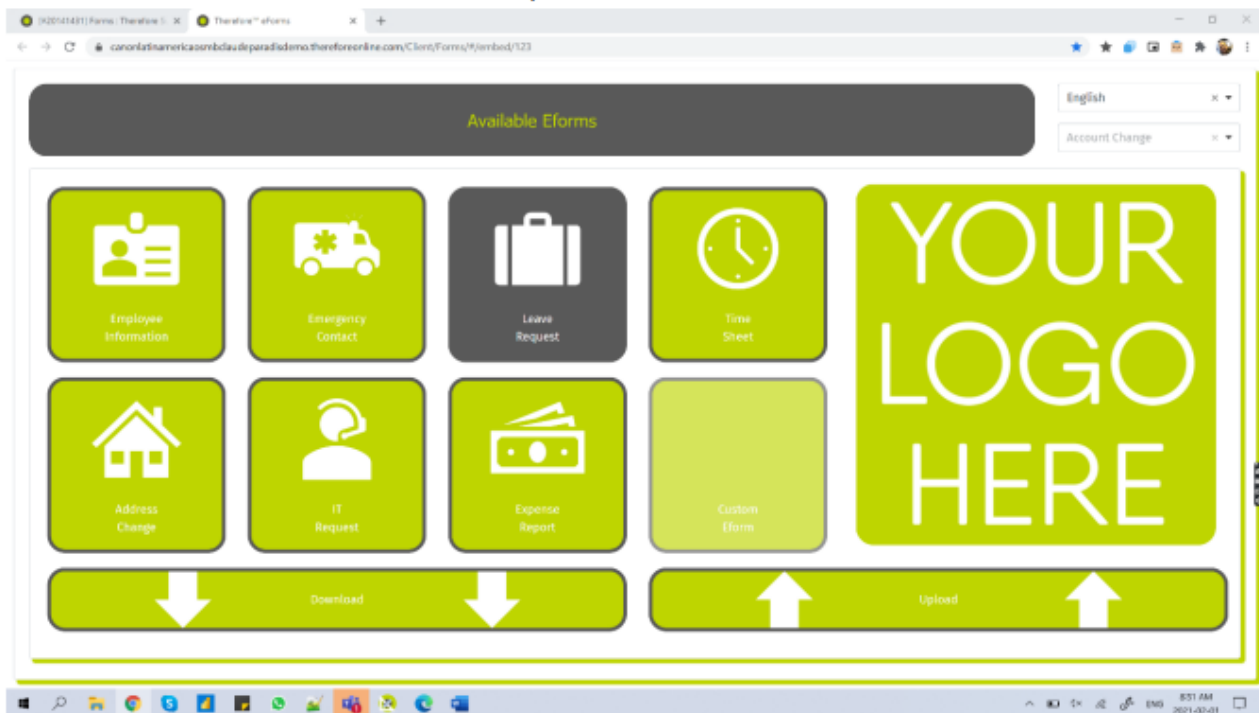
</style>
</head>
<body>

<h4 class="title3"> Upload section </h4>

</body>
</html>
```

This allows you to make visual changes in the same way as any web site using CSS.

You can now create an eForm like the one pictured below:



2.3.6 Scripting for Dynamic Color Changing

The following script can be added in the Conditional tab of any component to trigger a dynamic change.

Referring to Div of the text component

Text Field

aaa

```
if (data.textField === 'aaa')
{instance.element.style.backgroundColor = 'red'}
```

Referring to components[0] as component index in eForm

Text Field

aaa

```
if (data.textField === 'aaa')
{instance.root.components[0].element.style.backgroundColor = 'green'}
```

2.3.7 Color-Changing Background Using CSS

CSS animations can be used to make an eForm in which the background changes color.

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  animation: colorchange 5s infinite;
}
@keyframes colorchange
{
  0% {background: #f0f0f0;}
  50% {background: #595959;}
  100% {background: #f0f0f0;}
}
</style>
</head>
</html>
```

2.4 Grid

2.4.1 Layout

When using a **Data Grid** or an **Edit Grid**, you can incorporate a **Column** or a **Panel** component to add multiple fields with different positions in your grid.

Columns Component

Display API Conditional Logic Layout

Label *

Columns

☐ Hide Label

Label Position

Top

Placeholder

Placeholder

Column Properties

	Width	Offset	Push	Pull	
☰	1	0	0	0	⊙
☰	2	0	0	0	⊙
☰	3	0	0	0	⊙
☰	4	0	0	0	⊙
☰	5	0	0	0	⊙

Preview

Text Field

Text Field

Date / Time

Text Field

Text Field

yyyy-M

📅

Text Field

Date / Time

yyyy-MM-dd hh:mm i

📅

Save

Cancel

Remove

Data Grid

Date / Time

2020-12-17 9:27 AM

📅

⊙

+ Add Another

Submit

Data Grid

Columns

Text Field

Text Field

Date / Time

Text Field

⊙

Text Field

Text Field

yyyy-MM-dd hh:mm a

📅

Text Field

Date / Time

yyyy-MM-dd hh:mm a

📅

+ Add Another

2.4.2 Sum & Calculations

To calculate a sum or any calculation inside a grid, you can use the **row.** option.

JavaScript

```
1 value = row.quantity * row.unitprice;
```

Enter custom javascript code.

Example:

```
value = data.a + data.b + data.c;
```

Example formula for row price sum:

```
value = row.quantity4 * row.unitPrice4;
```

quantity4 = API Name of column with quantity

unitPrice4 = API Name of column with unit price

```
value = data.editGrid2.reduce((a, b) => a + (b['totalPrice4'] || 0), 0);
```

editGrid2 = API Name of Edit Grid Component

totalPrice4 = API Name of column to sum up

2.4.3 Save Collapse

If you need to hide the contents of a grid when the eForm is produced as a PDF, select the checkbox highlighted below.

Edit Grid Component

Display Templates Data Validation API Conditional Logic Layout

☒ Collapse grid when saving to PDF

Header Template

```

1 <div class="row">
2   {% util.eachComponent(components, function(component) { %}
3     {% if (!component.hasOwnProperty('tableView') || component.tableView) { %}
4       <div class="col-sm-2">{{ component.label }}</div>
5     {% } %}
6   {% } %}
7 </div>
  
```

Two available variables. "value" is the array of row data and "components" is the array of components in the grid.

Preview

Edit Grid

A	B	C	D
---	---	---	---

+ Add Another

Save Cancel Remove

2.4.4 Hide Column in Grid Render for End User

In this scenario, an expense report is being filled out. The user needs to input a lot of information when filling out the form, but the final table result should contain only the key value.

The user needs to enter data for all these fields: **Date, Type, Description, Currency, Other Currency, Exchange Rate, Amount, Tax, etc...** However, the final table result should display only Date, Type, Description and Amount.

Détails remboursement

Date * 2021-05-12

Type * Avion

Description * test

Devise * Autre

Détail de la devise

Code de la devise CAD

Valeur de la devise 0.770

Image

Drop files to attach, or [browse](#)

Montant à rembourser 770.00

Montant TTC * 1000.00

TVA * 150.00

Montant HT * 850.00

Save Cancel

If you need a column to appear when the end user is creating a row, but not appear in the final result, you can do the following:

Date	Type	Description	Montant à rembourser
2021-05-12	Air Travel	test	770

In your **Grid** component, in the **Template** tab, you need to modify both the headers and the row.

Edit Grid Component

Display **Templates** Data Validation API Conditional Logic

Layout

☐ Collapse grid when saving to PDF

Header Template

```

1 <div class="row">
2   {% util.eachComponent(components, function(component) { %}
3   {% if (!component.hasOwnProperty('tableView') || component.tableView) { %}
4     <div class="col-sm-2">{{ component.label }}</div>
5   {% } %}
6   {% } %}
7 </div>

```

Two available variables. "value" is the array of row data and "components" is the array of components in the grid.

Row Template

```

1 <div class="row">
2   {% util.eachComponent(components, function(component) { %}
3   {% if (!component.hasOwnProperty('tableView') || component.tableView) { %}
4     <div class="col-sm-2">
5       {{ getView(component, row[component.key]) }}
6     </div>
7   {% } %}
8   {% } %}
9   {% if (!instance.options.readOnly) { %}
10    <div class="col-sm-2">
11      <div class="btn-group pull-right">
12        <button class="btn btn-default btn-light btn-sm editRow">Edit Row</button>
13        <button class="btn btn-danger btn-sm removeRow">Remove Row</button>
14      </div>

```

Two available variables. "row" is an object of one row's data and "components" is the array of components in the grid. To add click events, add the classes "editRow" and "removeRow" to elements.

Preview

Item ID	Description	Quantity	Unit Price	Total Price
+ Add Another				

Save Cancel Remove

In this example, the table contains multiple columns that need to be hidden in the user interface when rows are added.

All of the components that have API in yellow below would be hidden. Only the components that are not listed will be shown in the result table.

The Headers (the part you need to add is highlighted in yellow)

```

<div class="row">
  {% util.eachComponent(components, function(component) { %}
  {% if (!component.hasOwnProperty('tableView') || component.tableView) { %}

```

```

{% if (component.key !== 'select') { %}
{% if (component.key !== 'devise') { %}
{% if (component.key !== 'A4') { %}
{% if (component.key !== 'A3') { %}
{% if (component.key !== 'A') { %}
{% if (component.key !== 'upload2') { %}
{% if (component.key !== 'monnaie2') { %}
{% if (component.key !== 'B') { %}
<div class="col-sm-2">{{ component.label }}</div>
{% } %}
{% } %}
{% } %}
{% } %}
{% } %}
{% } %}
{% } %}
{% } %}
{% } %}
{% } %}
{% } %}
{% } %}
{% }) %}
</div>

```

The Row (the part you need to add highlighted in yellow)

```

<div class="row">
  {%util.eachComponent(components, function(component) { %}
  {% if (component.key !== 'select') { %}
  {% if (component.key !== 'devise') { %}
  {% if (component.key !== 'A4') { %}
  {% if (component.key !== 'A3') { %}
  {% if (component.key !== 'A') { %}
  {% if (component.key !== 'upload2') { %}
  {% if (component.key !== 'monnaie2') { %}
  {% if (component.key !== 'B') { %}
  <div class="col-sm-2">
    {{ row[component.key] }}
  </div>
  {% } %}
  {% } %}
  {% } %}
  {% } %}
  {% } %}
  {% } %}
  {% } %}
  {% } %}
  {% }) %}
  <div class="col-sm-2">
    <div class="btn-group pull-right">
      <div class="btn btn-default btn-sm editRow"><i class="fa fa-edit"></i></div>
      <div class="btn btn-danger btn-sm removeRow"><i class="fa fa-trash"></i></div>
    </div>
  </div>
</div>

```

This needs to be done for all columns you want to hide

2.5 Image

2.5.1 Change HTML According to Trigger

Use the script below to change an image in reaction to a trigger. This script needs to be in the **Calculation** tab of the **HTML component**.

In the example below, the trigger is a **Select component** with api = select and option 1 and 2.



```
if (data.select === '1') {
  instance.htmlElement.innerHTML = "<img
src='https://storage.googleapis.com/therefore_hr/Canon-Therefore.jpg' width=50%
height=50%>";
}
else if (data.select === '2') {
  instance.htmlElement.innerHTML = "<img
src='https://storage.googleapis.com/therefore_hr/therefore%20logo.png' width=50%
height=50%>";
}
```

Important here is that a loop should always contain either a manual or a wait task to prevent countless executions in a very short timeframe.

EXAMPLE:

If you select Therefore™ Online from the option below, the Therefore™ Online logo is shown. Otherwise, the regular Therefore™ logo is shown.



Select

Therefore Online



Submit

2.5.2 Base 64

You can convert any image to a base 64 line using a website such as <https://www.base64-image.de/>

Use this code in the **HTML component**:

```

```

HTML Element Component

Display

Data

Validation

API

Conditional

Logic

Layout

Label ? *

HTML

☐ Hide Label ?

Label Position ?

Top

×

▼

HTML Tag ?

p

CSS Class ?

CSS Class

Attributes ?

Attribute	Value
+ Add Another	

Content ?

```

1 
9 />
10

```

Preview

Save

Cancel

Remove

2.6 Page Break

2.6.1 In a PDF

When an eForm is submitted, a PDF version of the form is created in Therefore™. Use "pagebreak-here" in the appropriate field to manually set where the page break should occur.

This needs to be added to the component that should be before the page break.

Select Component

Display

Data

Validation

API

Conditional

Logic

Layout

Label ? *

Select

☐ Hide Label ?

Label Position ?

Top

Placeholder ?

Placeholder

Description ?

Description for this field.

Tooltip ?

To add a tooltip to this field, enter text here.

Error Label ?

Error Label

Custom CSS Class ?

pagebreak-here

How to optimize page breaks in a PDF

Page breaks can be customized using the **HTML Element** eform component. Use defined dedicated classes to specify where page breaks should be located. Please click the following link to find more information about CSS rules:

https://www.w3schools.com/cssref/pr_print_pageba.asp

The default is to set a page break before, inside, or after the element which has the special dedicated class defined inside of the HTML Element.

It is also possible to protect some elements in order to prevent page breaks using CSS:

```
@media print {
  div {
    page-break-inside: avoid;
  }
}
```

More information about preventing page breaks can be found here:

<https://www.codesdope.com/blog/article/css-page-break-inside/>

An example of setting page breaks using the HTML element component is demonstrated in the following screenshot:



```
Content ?
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <style>
5   div > p.the-page-break-before {
6     break-before: always;
7     page-break-before: always;
8     /*background: green;*/
9   }
10
11   div > p.the-page-break-after {
12     break-after: always;
13     page-break-after: always;
14     /*background: yellow;*/
15   }
16
17   /*ul.editgrid-listgroup li:nth-child(2n),*/
18   ul.editgrid-listgroup div.divpagebreak-here:nth-child(2n) {
19     break-after: always;
20     page-break-after: always;
21     /*background: #bed500;*/
22   }
23
24   @page {
25     padding-top: 25px;
26     margin-top: 25px;
27   }
28 </style>
29 </head>
30
31
```

Note:

It is easier to develop style-sheet rules if the structure displayed in the browser is the same as the one of the PDF.

2.6.2 In HTML

This script will create a page break in an HTML component.

```
</p>  
  <div>  
    <div class="pagebreak-here"></div>  
    <div class="DIV1">...</div>  
  </div>  
</p>
```

Select Component

Display

Data

Validation

API

Conditional

Logic

Layout

Label ? *

Select

☐ Hide Label ?

Label Position ?

Top

×

▼

Placeholder ?

Placeholder

Description ?

Description for this field.

Tooltip ?

To add a tooltip to this field, enter text here.

Error Label ?

Error Label

Custom CSS Class ?

pagebreak-here

2.7 PDF

2.7.1 PDF customization

In a Therefore™ On-Premises installation, PDF customization can be configured on the server level. This means that the set customization is applied to all PDFs that are created from an eForm submission on this server. This method can be used to set branding colors or formatting server-wide.

The files that have to be edited in order to customize PDFs can be found in the following folder:

```
$SERVER\Program Files\Therefore\FormConvert\
```

First, custom.css must be enabled in the file titled 'Convert.Form.html'. The file contains information about which line should be uncommented in order to enable custom.css.

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <html>
4
5  <head>
6      <link rel='stylesheet' href='./shared/formio.full.min.css'>
7      <link rel='stylesheet' href='./shared/bootstrap.min.css'>
8      <link rel='stylesheet' href='./shared/the.css'>
9      <!-- uncomment next line to enable custom.css -->
10     <!-- <link rel='stylesheet' href='./custom.css'> -->
11     <script src='./scripts/formio.full.min.js'></script>
12     <script src='./scripts/the.js'></script>
13     <title>eForm Conversion</title>
14 </head>
15
16 <body>
17     <div id='formio'></div>
18     <div id='json-holder'></div>
19 </body>
20
21 </html>
22

```

Now, basic or complex rules for PDF customization can be specified in the file 'custom.css'.

Note: Custom CSS for specific eForms



Custom CSS can also be defined for specific eForms. To do so, define the rules in the 'Custom CSS' field of an HTML element and call it from the 'Custom CSS' field of other eForm components. This solution works both on Therefore™ On-Premises and Therefore™ Online.

2.8 Pre-populate field

2.8.1 Basic

A URL such as the one below will pre-populate an eForm when accessed via link.

<https://<server>/eForms/#/embed/10?apiKey1=Test&apiKey2=Test2>

If the data is coming from your Therefore™ category, use the structure shown below to use category field macros.

```
apiKey1=[Field1]&apiKey2=[Field2]
```

2.8.2 Advanced

Below is a complex example of a URL to prefill a Therefore™ eForm:

```
https://samptenant.thereforeonline.com/eForms/#/embed/23/?
allowanonymous=1&postaction=close&invoiceDate=[InvoiceDate]
&providerName=[VendorName]
&invoiceNumbe=[InvoiceNumber]&grossAmo=[GrossAmount]&uniqueNumber=[DocumentId]
&triggerJustificationLevel1=1
```

The URL is broken down as follows:

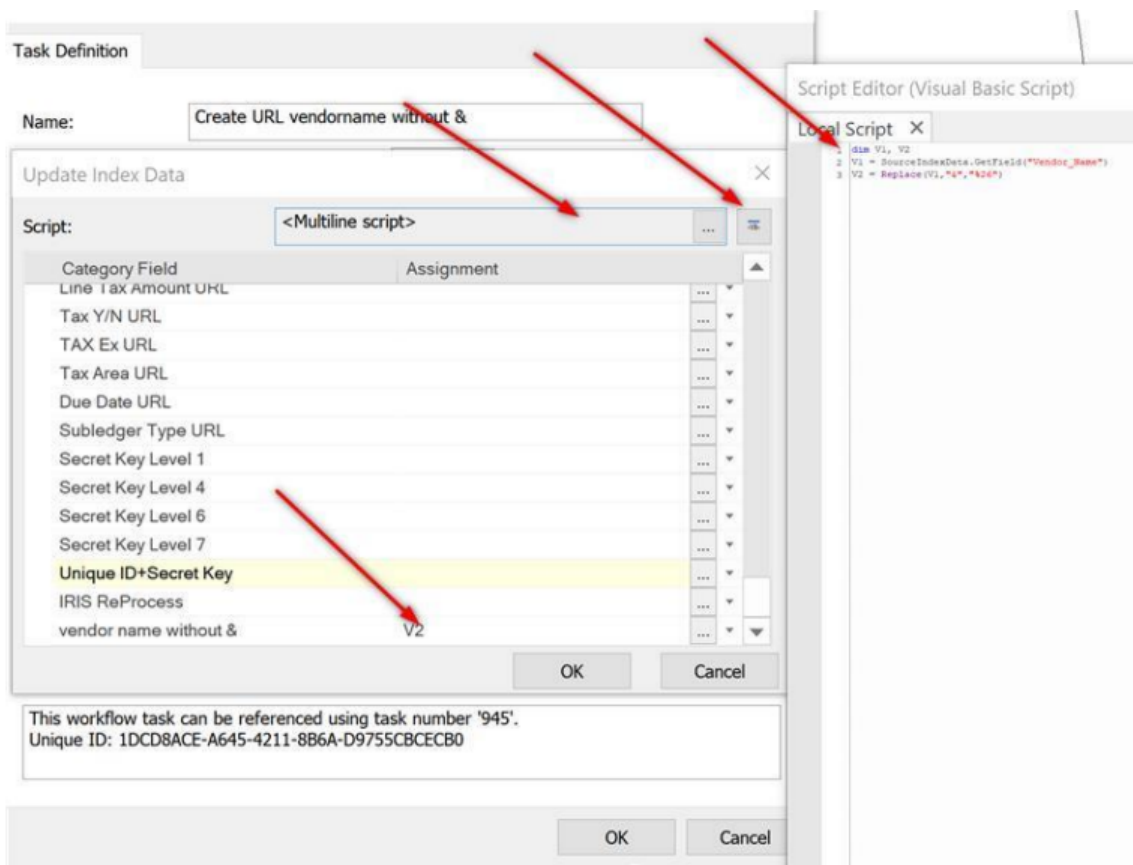
https://	Fix
samptenant	Tenant Name
.thereforeonline.com/eForms/#/embed/	Fix
23/	Eform Number
?allowanonymous=1	Authorized public user
&postaction=close	Close web browser tab after submission
&invoiceDate=[Invoice Date]	Populate field api XXX with Macro [XXX]
&providerName=[Vendor Name]	Populate field api XXX with Macro [XXX]
&invoiceNumbe=[Invoice Number]	Populate field api XXX with Macro [XXX]
&grossAmo=[Gross Amount]	Populate field api XXX with Macro [XXX]
&uniqueNumber=[Document Id]	Populate field api XXX with Macro [XXX]
&triggerJustificationLevel1=1	Populate field api XXX with Macro [XXX]

2.8.3 Error Handling

In some cases, there may be issues using characters that cannot exist in a URL without changing the URL function. For example, the ampersand character (&) cannot be used; if you have a text with character (e.g. Vendor Name "AT&T"), it will only show "AT" and there may be other issues with the eForm.

In this case, you must create a hidden data containing the replacement of this Vendor Name.

The code to replace "&" in a URL is **%26**. The following code will simply take "AT&T" and convert it to "AT%26T". This can then be used in the URL.



```

dim V1, V2
V1 = SourceIndexData.GetField("Vendor_Name")
V2 = Replace(V1,"&","%26")

```

2.8.4 Escaping Special Characters

The following script will correct all URL special characters at the same time.

```
function convert_text($text) {
    $t = $text;
    $specChars = array(
        ' ' => '%20', '!' => '%21', '"' => '%22',
        '#' => '%23', '$' => '%24', '%' => '%25',
        '&' => '%26', '\' => '%27', '(' => '%28',
        ')' => '%29', '*' => '%2A', '+' => '%2B',
        ',' => '%2C', '-' => '%2D', '.' => '%2E',
        '/' => '%2F', ':' => '%3A', ';' => '%3B',
        '<' => '%3C', '=' => '%3D', '>' => '%3E',
        '?' => '%3F', '@' => '%40', '[' => '%5B',
        '\\\' => '%5C', ']' => '%5D', '^' => '%5E',
        '_' => '%5F', '`' => '%60', '{' => '%7B',
        '|' => '%7C', '}' => '%7D', '~' => '%7E',
    );
    foreach ($specChars as $k => $v) {
        $t = str_replace($k, $v, $t);
    }
    return $t;
}
```

2.9 Web Call

2.9.1 Call Webservice from eForm

This script avoids two scenarios:

- Submitting an eForm before the REST call answer is returned.
- Keeping the REST call answer in a **Hidden** field to avoid an unnecessary REST call.



This component is mandatory for the script to work properly.

```

debugger; → optional - debug the code in the browser console debugger
var jsonData = data.hiddenTest; → check if the data is loaded already / do not
call the Web Service
every time
var htmlObj = document.getElementById('html_contain');
var url = 'https://mdn.github.io/learning-
area/javascript/ojs/json/superheroes.json'; → url for GET
call
if(data.callWebService === true && jsonData === undefined){ → only load again if
not loaded yet
fetch(url).then((response) => { → Web Service call
if(response.ok){
return response.json();
}else{
htmlObj.innerHTML = 'something wrong';
//throw new Error('Something went wrong');
}
}).then((jsonres) => { → process result of call
debugger;
data.hiddenTest=jsonres; → save the data to hiddenTest when loaded
if(jsonres.active){
// do more with the data, f.ex. populate data object

```

2.9.2 Call the Therefore™ WebAPI

The following solutions can be used for calling the Therefore™ WebAPI from Therefore™ eForms.

There are three options to call the Therefore™ WebAPI from Therefore™ eForms. The options available when calling third party APIs are out of our hands and not within the scope of this guide.

1. Basic Authentication

Basic Authentication is the easiest technical option. To use this option, make sure to create a user with very limited options regarding account security. The user should only be given enough permissions to do exactly what he is supposed to do.

This option is valid but dangerous in case of misconfiguration. Please also be aware that the credentials given to this user - including the password for this account - are then publically available. Because of the associated security risks we recommend using option 2 or 3.

2. Use the token of the user that is currently signed in

This is the best approach for an authenticated form. Use the token of the authenticated user - whatever this user is allowed to see will be accessible in the eForm. This is the safest way to authenticate. Below, please find a code sample of how this can be done.

Please note that this option does not work with anonymous forms since the token is stripped down to only allow access to this one form and nothing else.

Code Example:

```
var xmlhttp = new XMLHttpRequest();
xmlhttp.open('POST',
'http://$SERVERNAME/theservice/v0001/restun/ExecuteSingleQuery', true);
//Token Authentication:
let theToken = '';
const loginInfo = JSON.parse(localStorage.getItem('loginInfo'));
if (loginInfo !== null) {
  theToken = loginInfo.token;
}
xmlhttp.setRequestHeader('Authorization', 'Bearer ' + theToken);
xmlhttp.setRequestHeader('Content-Type', 'application/json');
//Tenant:
//xmlhttp.setRequestHeader('TenantName', 'TENANT NAME');

let condition = '{CUSTOM CONDITION...}';
xmlhttp.send(condition);
xmlhttp.onreadystatechange = function () {
  if (xmlhttp.readyState == 4) {
    if (xmlhttp.status == 200) {
      PROCESS THE RESULT...
    }
  }
};
```

3. Custom JWT token

Use the custom JWT Token feature of Therefore™ to craft a special stripped down access token and use it in anonymous eForms.

Find a tutorial on how to create custom JWT Tokens here:

https://www.therefore.net/help/Online/en-us/sd_t_authmeth_admintasks_jwt.html

To create such a token for anonymous eForms please follow the documentation as provided. In addition, please set the following scope:

```
"urn:oauth:scope": "therefore_specific"
```

If "therefore_specific" is defined as a scope permissions need to be set as follows:

```
"the:<objecttype>": "<object_no>:permission"
```

Please find more information of permissions on the following pages.

List of permissions:

https://therefore.net/help/Online/en-us/AR/SDK/WebAPI/the_webapi_rbac.html

Permission levels and syntax:

https://therefore.net/help/Online/en-us/sd_r_access_authentication_jwt_permissions.html

The validity of the token could be long (years) so it does not have to be exchanged frequently.

Please note that this token is also public available and could be used outside the context of eForms to call other APIs. Still, due to the strict scope this should be an acceptable risk. In addition, all issued tokens can easily be rendered invalid in one step by changing the pre-shared key of the token issue in case someone a token might have been stolen or tampered with.

The following code is an example of such a custom token for anonymous access:

```
{
  "iss": https://<Issuer ID>.domain,
  "iat": 1684966685,
  "exp": 1716502688,
  "aud": "Customer ID",
  "urn:oauth:scope": "therefore_specific",

  "http://schemas.microsoft.com/ws/2008/06/identity/claims/windowsaccountname": "<account name>",
  "the:casedef": "9:256",
  "the:ctgry": "305:256"
}
```

iss: Issuer Id: "iss" must match the Issuer defined in the configuration in Therefore™.

aud: This must be the Customer ID

2.9.3 Get Data Back (Response) from REST Call

```
function processData () {
  if (this.status === 200) {
    jsonResponse = JSON.parse(this.responseText);
    textfield.value = jsonResponse.****; //update text field in UI
  }
}

var tenant = '';
var authString = '';
var method = '';
var baseUrl = '';
var endPoint = '';
var jsonResponse = '';
var xmlhttp = new XMLHttpRequest();
if (data.clientID) {
  var thereforeJson = '{"CaseNo":"' + data.clientID + '","IsAccessMaskNeeded":true}';
  xmlhttp.addEventListener('load', processData);
  xmlhttp.open(method, endPoint, true);
  xmlhttp.send();
}
```


2.9.4 Select Component from Web Service

Use the following method if you need a **Select component** to contain information from a REST call response.

Select Component

Display Data Validation API Conditional Logic Layout

Data Source Type: URL

Data Source URL: `https://restcountries.eu/rest/v2/all`

☒ Lazy Load URL

Request Headers

Key	Value

+ Add Another

Data Path

The object path to the iterable items.

Value Property: `name`

The selected item's property to save.

Preview

Country

Save Cancel Remove

The object path to the iterable items.

Value Property: `name`

The selected item's property to save.

☐ Disable limiting response

Search Query Name

Name of URL query parameter

The minimum amount of characters they must type before a search is made.

Filter Query

Sort Query: `3`

The sort query for results

Limit: `100`

Maximum number of items to view per page of results.

Item Template

1 `<span>[[ item.name ]]</span>`

2.10 Other

2.10.1 Dependent Lookup

Follow these steps to create a dependent lookup:

Table Lookup

Text Field

Text Field

Dependent Lookup

Supplier

Product

Submit

ProductID	ProductName	SupplierID
1	Chai	1
2	Chang	1
3	Aniseed Syrup	1
4	Chef Anton's Cajun Seas...	2
5	Chef Anton's Gumbo Mix	2
6	Grandma's Boysenberry ...	3
7	Uncle Bob's Organic Drie...	3
8	Northwoods Cranberry S...	3
9	Mishi Kobe Niku	4
10	Ikura	4

Items per page 10 1 - 10 of 77 |< < > >| Close

The objective is to have a second lookup that will show only the item filtered by the previous lookup.

Table Lookup

9

Text Field

Mishi Kobe Niku

Text Field

4

Dependent Lookup

Supplier

4

Product

Submit

Table Lookup

9

Text Field

Mishi Kobe Niku

Text Field

4

Dependent Lookup

Supplier

4

Product

Submit

ProductID	ProductName	SupplierID
9	Mishi Kobe Niku	4
10	Ikura	4
74	Longlife Tofu	4

Items per page 10 1 - 3 of 3

Close

This shows that only items related to supplier #4 are available.

This was done with a logic on the "Supplier" **Text Field**:

The screenshot shows the 'Text Field Component' configuration window with the 'Logic' tab selected. The 'Advanced Logic' section indicates '1 Advanced Logic Configured'. The 'Logic Name' is 'Fill Supplier'. The 'Trigger' section shows the 'Container' as 'Type' and the 'Type' as 'Javascript'. The 'Text Area' contains the code: `result = data['textField3'].value;`, which is highlighted with a red box. The 'Preview' pane on the right shows a text input field labeled 'Supplier' with 'Save', 'Cancel', and 'Remove' buttons below it.

The screenshot shows the 'Actions' configuration window. The '1 actions' section shows an 'Action Name' of 'Fill' and a 'Type' of 'Value'. The 'Value (Javascript)' section contains the code: `value = data.textfield3;`, which is highlighted with a red box. The 'Preview' pane on the right is empty. At the bottom, there are 'Save Action', 'Cancel', and '+ Add Action' buttons.

2.10.2 Do Not Save PDF

Use the following method if you want the index data of an eForm to be created/saved in Therefore™, but not the PDF file.

eForms Indexing Profile (Fund Approval)

Category: Fund Approval (Open)

Script: |

Auto-append mode: Category default

Category Field	Assignment
CompanyNo	
PM/PENo	
to be deleted	
Request For email	
FA Approval Number	
Brand	Json.GetValue("data.e82gxm")
Approved By	
FA Amount	Json.GetValue("data.fundAmount")
Approval Balance	Json.GetValue("data.fundAmount")
Status	
PM Amount	Json.GetValue("data.pmamount")
GP Amount	Json.GetValue("data.gpamount")
Vendor Claim Amount	Json.GetValue("data.vendorClaimAmou...")

Test... ☐ Disable automatic evaluation OK Cancel

Objects and Functions

- eForm Object
 - IncludeFormPDF
- Helpers
 - AdjustClient TimeDifference
 - Date
 - DecimalAdd
 - DecimalCmp
 - DecimalDiv
 - DecimalMul
 - DecimalSub
 - ToDate
 - ToDateTime
 - ToDecimal
 - ToFloat
- Index Data Object
 - GetCategoryName
 - GetField
 - GetField...

Script Editor (JavaScript)

Local Script X

```
1 EForm.IncludeFormPDF = false
```

2.10.3 PDF Page Width

It is possible to set the PDF page size and orientation on the eform level. Each and every eForm can have a different size and orientation.

This can be accomplished via the **HTML Element component** on the eform itself:

Content ?

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4 <style>
5 @page {
6     size: A3 portrait !important;
7     /*size: A4 portrait !important;*/
8     /*size: A4 landscape !important;*/
9 }
10 </style>
11 </head>
12 </html>

```

The `@page` CSS at-rule is used to modify some CSS properties when printing a document. When the PDF is generated, this page width will be also be applied. More info can be found under the link below:

<https://developer.mozilla.org/en-US/docs/Web/CSS/@page>

The PDF can even have a custom width and height as demonstrated in the following example:

```

@page {
    size: 4in 6in landscape;
}

```

2.10.4 Using script to Hide/Disable

Use this script to hide/disable any field in your eForm, including hidden fields.

```

form.components.find(c => c.component.key === 'select2').component.disabled =
true;
form.components.find(c => c.component.key === 'dateTime2').component.disabled =
true;
form.redraw();

```

`form.redraw()` is not always required, depending on the usage scenario.

2.10.5 Hide field on Multiple Conditions

If a field needs to be hidden with multiple triggers use **Logic**.

The screenshot shows the 'Text Field Component' configuration window with the 'Logic' tab selected. The 'Advanced Logic' section indicates '1 Advanced Logic Configured'. The 'Logic Name' is 'Check'. The 'Trigger' section shows the 'Container Type' as 'Javascript'. The 'Text Area' contains the code: `1 result = data.checkA === true && data.checkB === true;`. A note at the bottom states: 'row', 'data', and 'component' variables are available. Return 'result'. To the right, a 'Preview' window shows a text field.

The screenshot shows the 'Text Field Component' configuration window with the 'Actions' tab selected. The 'Actions' section indicates '1 actions'. The 'Action Name' is 'Hide'. The 'Type' is 'Property'. The 'Component Property' is 'Disabled'. The 'Set State' is 'True'. At the bottom, there are buttons for 'Save Action', 'Cancel', and '+ Add Action'.

2.10.6 Hide in PDF

It is possible to hide components in the PDF rendering of an eForm. To do so, open the eForm in Edit mode, click 'Edit' for the component that should be hidden and enter 'hide-in-pdf' under 'Custom CSS Class'. This will always hide the component.

To hide a component under certain conditions, edit the component, go to the 'Data' tab, and enter the following Java script in the 'Calculated Value' field. Instead of 'true', use the value that should cause the field to be hidden.

```
value = data[component.key];
if (value === 'true') {
instance.element.classList.add('hide-in-pdf');
} else {
instance.element.classList.remove('hide-in-pdf');
}
```

- It is also possible to hide different components by entering the following script under 'Calculated Value'. This script will hide the component 'SensitiveInformation' if its value is 'true'.

```
value = data[component.key];
let componentToHide = instance.root.components.find(c => c.key ===
'SensitiveInformation');
if (value === 'true') {
componentToHide.element.classList.add('hide-in-pdf');
} else {
componentToHide.element.classList.remove('hide-in-pdf');
}
```

- Instead of adding a script under 'Calculated Value' for multiple components a helper component can be added. For example, use a Text Component and set it to hidden. Add a 'Custom Default Value' in which it is possible to check multiple components and hide them.

2.10.7 Wizard eForm: Carry Data Between Pages

Use this method to carry a value from one page to another in a wizard-type eForm.

On page 1 of the eForm, there is a table lookup (**User Number**) with 2 related data (**Data 1** and **Data 2**).

Page 1

Page 2

Page 3

User Number

11

...

Data 1

choigek.seew

Data2

choigek.seew@therefore.net

Cancel

Next

On page 2 of the eForm, there is a field that needs to contain one of the related data (**Data 2**). Use Javascript as shown in the screenshot below.

Page 1 Page 2 Page 3

Data2

choigek.seew@therefore.net

Cancel Previous Next

Calculated Value

The following variables are available in all scripts.

form	The complete form JSON object
submission	The complete submission object.
data	The complete submission data object.
row	Contextual "row" data, used within DataGrid, EditGrid, and Container components
component	The current component JSON
instance	The current component instance.
value	The current value of the component.
moment	The moment.js library for date manipulation.
_	An instance of Lodash .
utils	An instance of the FormioUtils object.
util	An alias for "utils".

JavaScript

```
1 value-data.data2;
```

On page 3 of the eForm there are all three fields.

Page 1 Page 2 Page 3

Table Lookup

11

data1 page 3

choigek.seew

data2 page 3

choigek.seew@therefore.net

Cancel Previous Submit

Use Javascript as shown in the screenshot below. The same concept applies to both the lookup field and data field.

Calculated Value

The following variables are available in all scripts.

form	The complete form JSON object
submission	The complete submission object.
data	The complete submission data object.
row	Contextual "row" data, used within DataGrid, EditGrid, and Container components
component	The current component JSON
instance	The current component instance.
value	The current value of the component.
moment	The moment.js library for date manipulation.
_	An instance of Lodash .
utils	An instance of the FormioUtils object.
util	An alias for "utils".

JavaScript

1 value=data.tableLookup;

Calculated Value

The following variables are available in all scripts.

form	The complete form JSON object
submission	The complete submission object.
data	The complete submission data object.
row	Contextual "row" data, used within DataGrid, EditGrid, and Container components
component	The current component JSON
instance	The current component instance.
value	The current value of the component.
moment	The moment.js library for date manipulation.
_	An instance of Lodash .
utils	An instance of the FormioUtils object.
util	An alias for "utils".

JavaScript

1 value=data.data1;

* All fields can be hidden or visible and will still work properly.